HACKNYX WEB EXPLOITATION WORKSHOP

BREAKING THE WEB

FROM ZERO TO HERO

- → 1hour 30min hands-on workshop no experience needed
- → Target: LEKIR vulnerable on purpose, so we can break it
- → By the end: you'll have executed real commands on a real web app

QR and link for repository

https://github.com/k33ps77-rgb/hacknyx-breaking-the-web/tree/main

GETTING STARTED

Environment Setup

Scan the QR code or visit the tunnel URL on screen

Log in with the shared credentials

Keep browser DevTools open the whole session (F12)

If it looks broken, just refresh probably not you

INTRO OF ME?

$ whoami

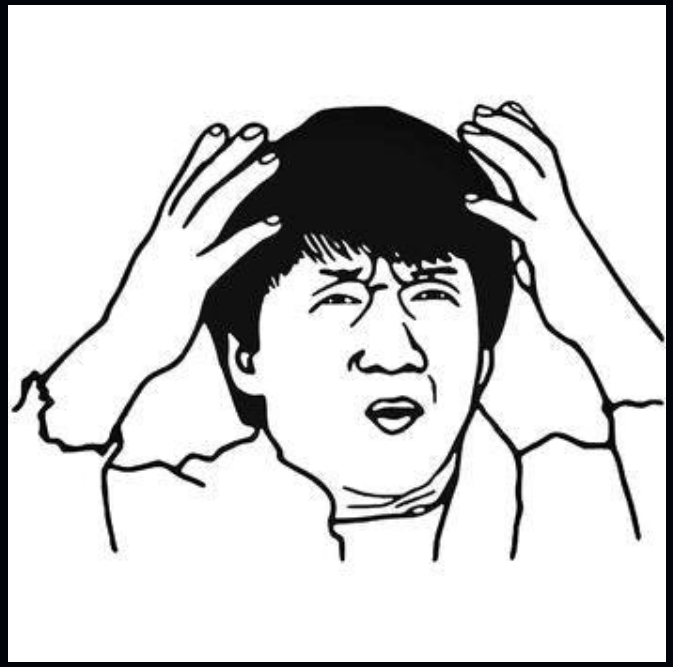
>>> Thaqif , Semester 5

$ background

>>> cybersecurity student , CTF rookie , newbie + skill issue

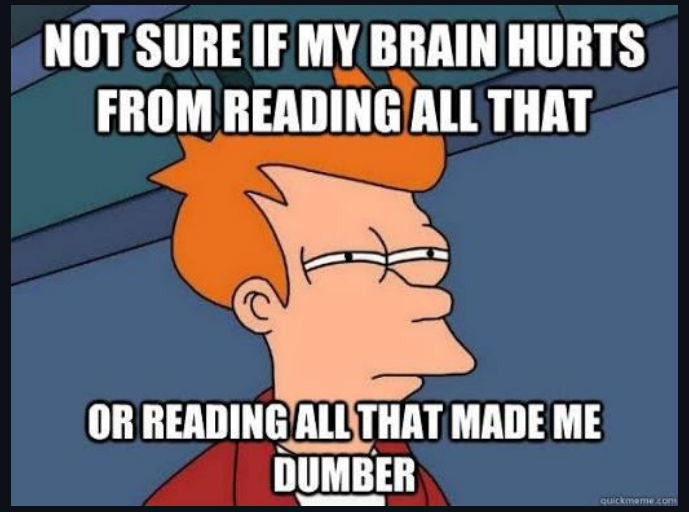
$ quote? Yea sure

>>> Passwords are like underwear: don't let people see it, change it very often, and you shouldn't share it with strangers.



What Is Web Exploitation?

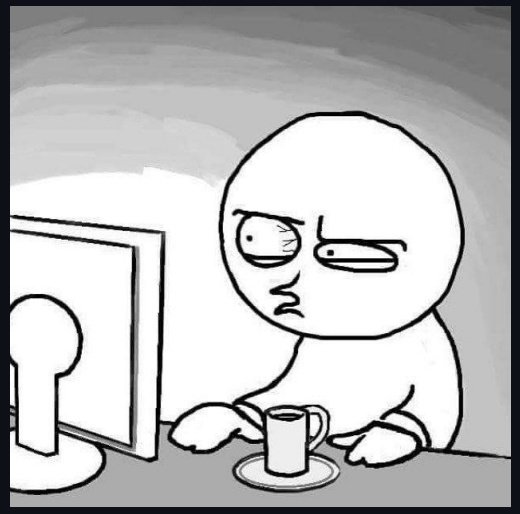
- → A website is just a server answering requests nothing scary
- → Browser sends a request → server sends back a response
- → Bugs happen when the server trusts the client way too much
- → Today's path: IDOR → SQLi → XSS → Cookies → File Upload (RCE)



What Is IDOR?

- → **Insecure Direct Object Reference** a fancy name for a simple mistake
- → An app shows you a record by its ID number, like an invoice or payslip
- → The bug: the app never checks if that record actually belongs to you
- → If you can guess or change the ID, you can see someone else's data

you right now →



IDOR → Walk the Numbers

```
$ open idor.php?statement=006


try these next:
  statement=003
  statement=014
  statement=018
  statement=064
  statement=467
```

- → No login bypass or fancy tools needed
- → Just curiosity and arithmetic
- → Change the ID, see what comes back

CHAPTER 2 · CONCEPT

What Is SQL Injection?

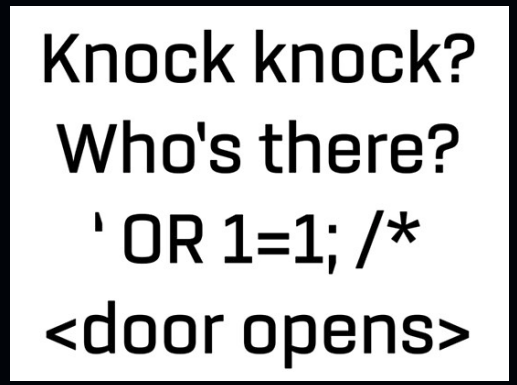
- → **Structured Query Language Injection (SQLI)**
- → The app builds a database question directly out of your input
- → Normally it asks: "find the user with this exact name"
- → Sneak in your own punctuation, and you change the question itself
- → This can bypass logins entirely, or dump the whole database

one character can break the whole query

SQL Injection → Break the Query

```
$ login as: 1' OR '1'='1
→ query logic broken, login bypassed

$ search: 1' UNION SELECT 1,
    user_name, user_password
    FROM user -- -
→ credentials dumped
```



- → A single quote is usually the first thing worth testing
- → The OR trick breaks the login logic wide open
- → UNION SELECT pulls extra data straight out of the database

What Is XSS?

- → **Cross-Site Scripting** when a page repeats your words back as code
- → You type something, the page shows it back to you (or to others)
- → If it forgets to treat that text as plain text, your input runs as a script
- → A real attacker can steal cookies, redirect users, or act on their behalf

<script>

the page just... runs it

Reflected XSS → Pop the Box

```
$ search:
"><script>alert('pwned')</script>
→ browser executes it as real code
→ an alert box pops up on the page
```

- → Your input becomes code the browser happily executes
- → A real attacker could steal cookies or redirect victims with this
- → Small payload, big consequences

What Is Cookie Manipulation?

- → Cookies are small notes your browser keeps and sends with every request
- → Some apps store a trust decision in that note like your role
- → The app just believes whatever the note says, no double-checking
- → Edit the note yourself, and you rewrite what the app believes about you

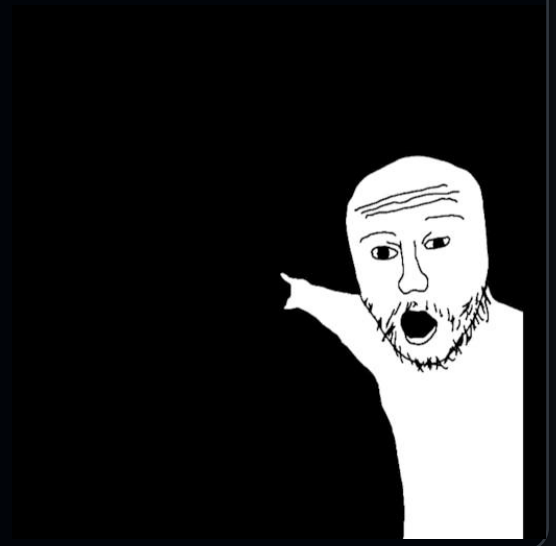
```
user_role = admin
```

Cookie Manipulation → Rewrite Trust

```
$ devtools → application → cookies

user_role: user
    ↓ edit
user_role: admin

$ refresh
→ congrats, admin.
```



- → DevTools → Application → Cookies
- → Find the trust value and change it
- → Refresh the page
- → Nothing was "hacked" the app simply believed you

What Is Insecure File Upload?

- → An upload form is supposed to only accept safe files, like images
- → The problem: it often only checks what the browser claims the file is
- → It never actually looks at what the file really contains
- → Sneak in a disguised script, and the server may run it for you

shell.php
Content-Type: image/jpeg

File Upload → RCE

```
$ upload shell.php
   Content-Type: spoofed to image/jpeg
→ intercepted + edited live in Burp Suite

$ curl target/uploads/shell.php?cmd=id
uid=33(www-data) gid=33(www-data)

>> RCE ACHIEVED
```

- → Upload a PHP web shell disguised with a spoofed Content-Type
- → Use Burp Suite to intercept and edit the upload request live
- → Trigger the shell through the browser and run real commands
- → We didn’t change the file — we changed what the browser claimed it was

WRAP-UP

THANK YOU

- → Questions and discussion no judgment zone
- → Keep exploring, keep breaking things safely
- → Next stops: picoCTF, CyLabs, HackTheBox Starting Point